Frontier Models as Weight Editors for Gemma 4 E4B

A gradient-free weight edit cuts a coding agent's escape-code errors from 15 % to 3 % in live runs, and shows what such fixes must also keep

Abstract

Can frontier models change a coding agent by writing its weights without gradients? We target Gemma 4 E4B's habit of copying escape codes into the text it wants to replace. In a test fixed in advance on 320 recorded edit calls from 69 held-out repositories, three edits cut this defect by 23–75 % against the unedited model, while size-matched random edits left it unchanged. Each also lost 3–11 of 213 previously working calls in the worse of two generation runs, where our preservation rule allowed 2 per run. One selected edit, served as a LoRA adapter on 4-bit E4B, cut the defect from 15 % to 3 % of live edit calls, and from 7 % to 1 % with thinking (reasoning before each action) on, in replications planned before their runs. Gradient training removed more artefacts but failed preservation too. We detected no solved-task benefit: the harness already repairs many escaped copies. Post-hoc rescoring without that repair shrinks the working-call subset; two edits preserve it and raise the share of old texts found by 4 and 7 points. We release tools, tasks and recorded calls.

1. Introduction

Gemma 4 E4B is a small open model (about 4.5 billion effective parameters, 42 layers). In the Gemma 4 Developer Agent Competition on Kaggle, a *harness* gives it tools to read and edit a Python repository and scores the hidden bug-fix tasks it resolves. To edit a file, the agent supplies *old text* to find and new text to replace it. Tool results show line breaks as the two characters `\n`, and quotes as `\"`; E4B often copies these codes into the old text instead of the real characters (Figure 1). We count a generated text as carrying an *escape artefact* when it contains either code and the file contains no such sequence.

(a) The file on disk

```
def total(items):
    return sum(item["price"] for item in items)
```

(b) What the model reads: the same text inside the tool result, written with escape codes

```
{"content": "def total(items):\n    return sum(item[\"price\"] for item in items)\n"}
```

(c) The old text the model writes when it edits this file

Unedited model

```
def total(items):\n    return sum(item[\"price\"] for item in items)
```

✗ not in the file; only a fallback that removes escapes rescues it

Edited model

```
def total(items):
    return sum(item["price"] for item in items)
```

✓ found in the file as written

escape code: \n stands for a line break, \" for a quote

Figure 1. A synthetic example: the unedited model copies escape codes; the edited model writes real characters.

The tool already repairs many such mistakes. Its *fallback* unescapes both old and new text when matching fails, then retries. It runs in every agent run and the scorer, rescuing 108 of 614 accepted edits on the training repositories. So we did not expect fewer artefacts to solve many more tasks. We chose this frequent, countable defect to ask a narrower question: can a frontier model change one behaviour while preserving calls that already work?

Writing weights directly is called *weight-space programming* [1]. We separate three questions (Figure 2): does behaviour change, are working calls preserved, and are more tasks solved?



Figure 2. Evidence map: replay, quantized serving and live runs; arrows mean “tested next”. Fable is Claude Fable 5.1; OOD-C is our new set of 186 tasks.

We add controlled editor experiments, live transfer and a gradient baseline. These fixed-budget tests do not establish editing’s limits.

2. Method

The study ran in two rounds; this paper reports round 2, which targeted escape artefacts, and summarises round 1 below. Each editor worked alone in a sandbox (Figure 4a) with weights and recorded calls from four training repositories: fastapi, rich, requests and httpx. It could read any weight but had no test data. One *forward-pass request* either regenerated old text on up to 128 calls, returning the share of old texts found (acceptance) and artefact rates, or read activations and log-probabilities on up to 32 texts. Claude Opus 5.5 and Claude Fable 5.1 had 25 requests each; GPT-6 Astra had 10. A separately registered (planned in advance) *informed* Opus run could read the earlier editors’ notebooks, answers and edits and had 10 requests. Each could submit one edit or none. *Gradient-free* means backpropagation and iterative optimisers were forbidden; closed-form fits, including least squares, were allowed. Two later Fable 5.2 runs, 10 requests each, tried an informed edit and repairing a gradient-trained adapter’s losses.

Round 1 targeted old-text matching generally. Opus and Fable had 10 requests each; a fixed recipe of ours, a scripted difference-of-means ablation, supplied a third edit. None beat matched random edits on 225 held-back training-repository calls or changed escape artefacts, motivating round 2.

Each edit changes one or more weight matrices W to W + BA, with thin matrices A and B: LoRA form [2] (Figure 3). Fable’s rank-2 edit acts in the last layer’s feed-forward output matrix. A reads two detectors from activations, one before an escaped line break and one before an escaped quote; B raises the scores of raw characters. Both are fitted in closed form. Opus instead repurposed 40 last-layer neurons it judged little used, switched on by a rank-1 “flag” written in layer 40 (second-to-last, numbered 0–41); informed Opus used 80 such neurons, and the withdrawn informed Fable 5.2 candidate 86.

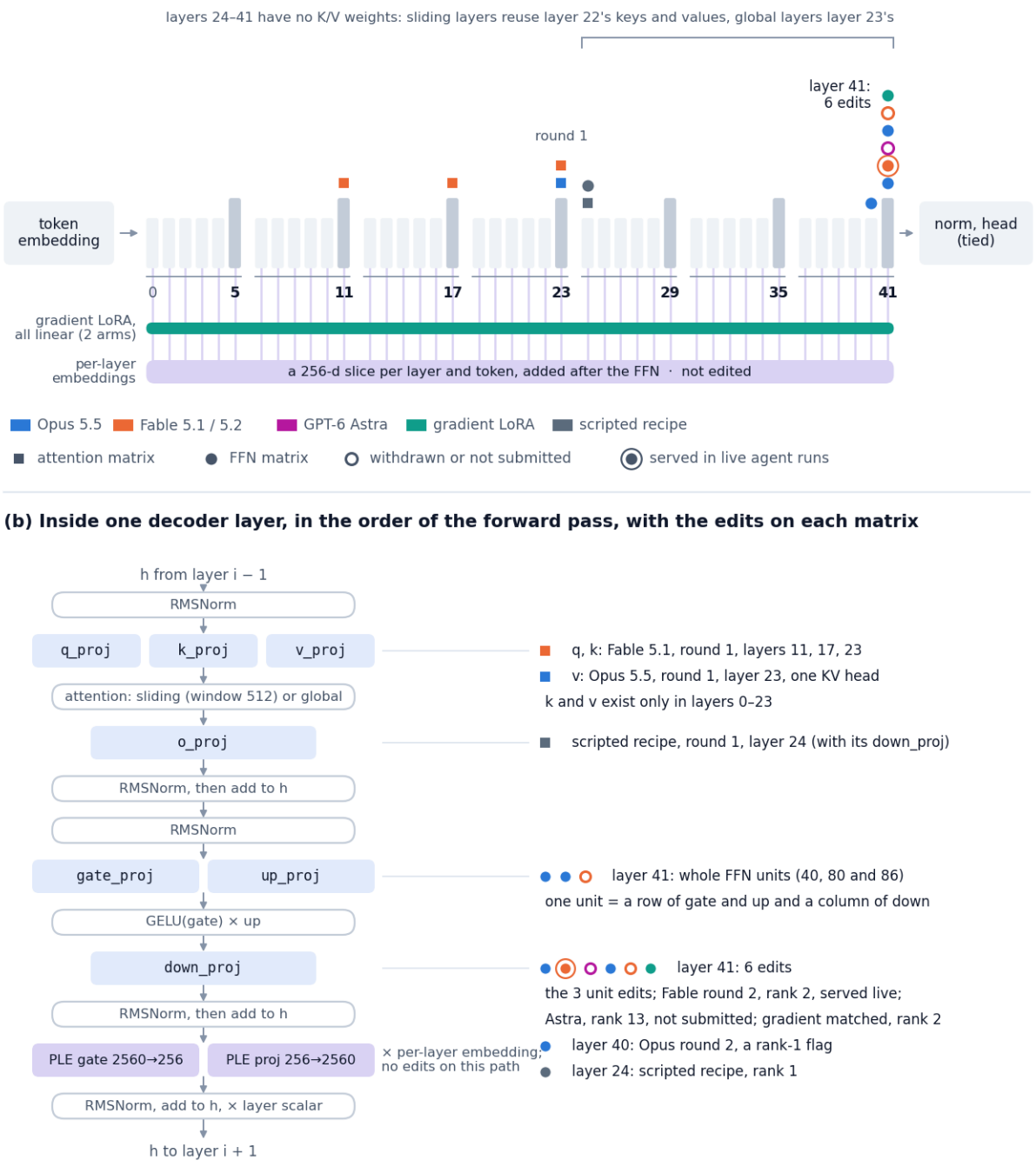


Figure 3. Where the edits sit in E4B: (a) the 42 layers, with each edit on its layer; (b) one layer, with the matrices each edit changes. Layers 24–41 reuse earlier layers' keys and values, so they have no key or value weights to edit.

We test each frozen edit by replay: the model regenerates old text from the original prompt and fixed preceding context (Figure 4b). *Acceptance* means a successful match under the tool's rules. The *gate*, a test fixed in advance [3], uses 320 calls from 101 tasks in 69 repositories disjoint from editor data. Each call has two *draws*, one greedy and one at temperature 0.6, giving 640 generations. Each edit faces the base and two random controls matched in location, rank and size (Frobenius norm of the weight change), but with random directions. A task's artefact rate averages its calls and both draws. It must fall against all three references in one-sided exact sign tests over tasks, Holm-corrected across

Opus and Fable; informed Opus was registered separately. Registration means a timestamped plan in the study log before testing. Three guards limit side effects:

- Task-averaged acceptance may fall at most 1 percentage point.
- *Working calls* are calls that succeeded when recorded and that the base model's replay also matched (213 per draw). At most 2 may be lost per draw (*negative flips* [11]); an accepted match is not a correct bug fix.
- Among accepted calls, *compile failures* in the edited Python file may rise at most 2 points. For this guard only, a second replay regenerates both texts, because the fallback changes both; this form replaced the registered one, which kept the recorded new text, before any edit ran on the test calls. It turns all three edits' compile verdicts from fail to pass but changes no overall verdict (full paper).

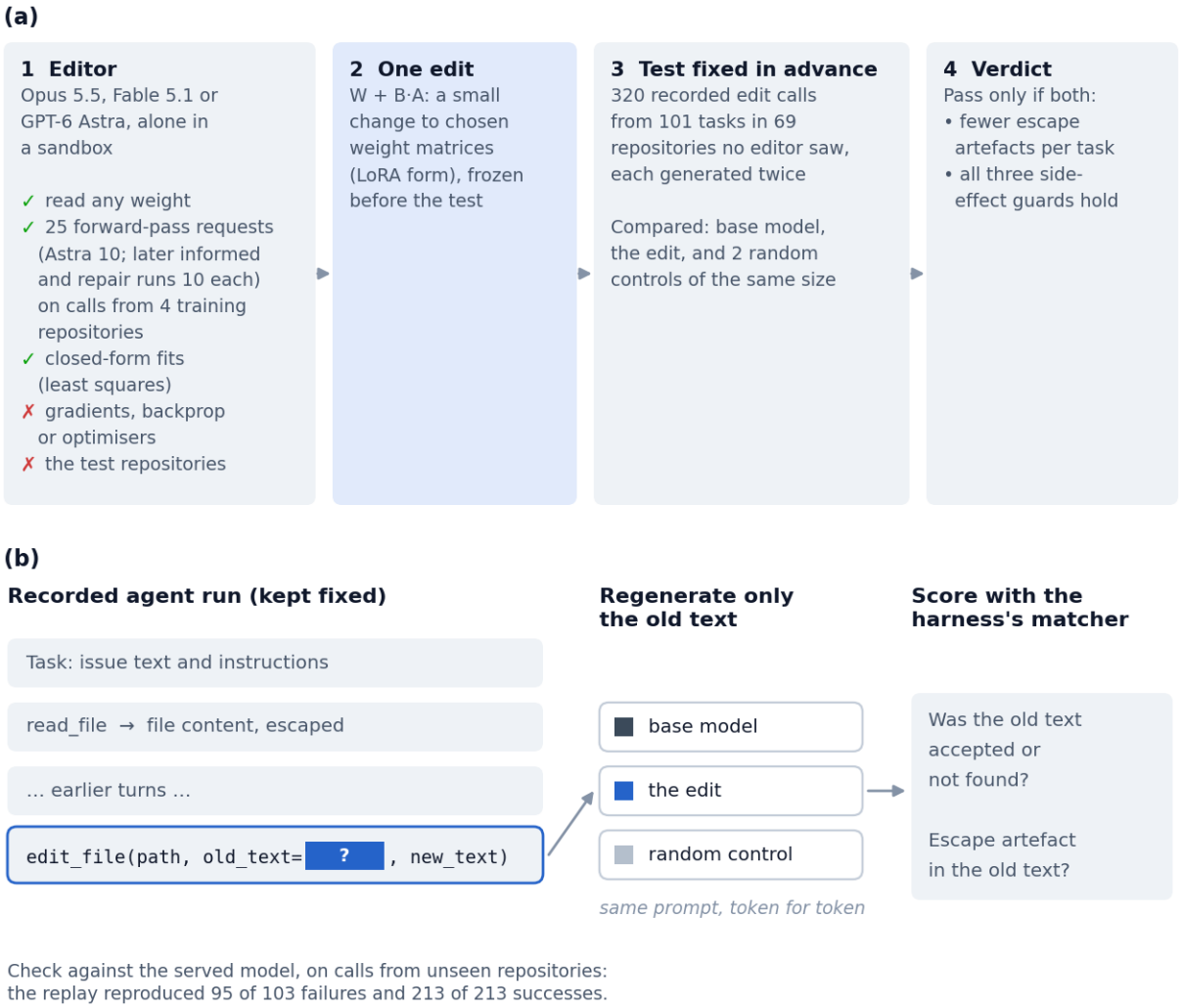


Figure 4. Editor access and frozen-context replay. Base replay reproduces 95/103 recorded matching failures and all 213 successes; four other calls failed otherwise, three through multiple matches.

OOD means out of distribution: repositories outside training. OOD-43 has 45 tasks from 17 repositories, 43 gradable; OOD-B has 82 from 70 repositories. Five E4B runs supplied 209 usable calls from OOD-43 and 111 from OOD-B: each has verified context, file state and old text. The first live test used 44 OOD-43 and 71 OOD-B tasks; the rest could not run or be graded on our setup. OOD-C, built from SWE-rebench [4, 5], adds 186 tasks from 148 repositories disjoint from both sets and training. Only 53 are *gradable*: the dataset's reference patch passes twice, the empty patch fails, and the reference passes again under Python 3.12. A task is *solved* when the agent's patch passes hidden tests. Live runs count calls with an artefact in either text, leaving out calls whose file state could not be rebuilt.

3. Results

3.1 Targeted change and preservation

All three submitted edits cut artefacts and failed preservation (Figures 5, 7a). Of 640 base generations, 100 carried an artefact; Opus left 77, Fable 47 and informed Opus 25: reductions of 23, 53 and 75 %. Each beat the base and both controls ($p \leq 0.011$); controls left 99–102. Yet the edits lost 3, 5 and 11 working calls in their worse draw, exceeding 2. Fable narrowly failed acceptance (−1.01 points), and informed Opus failed it too.

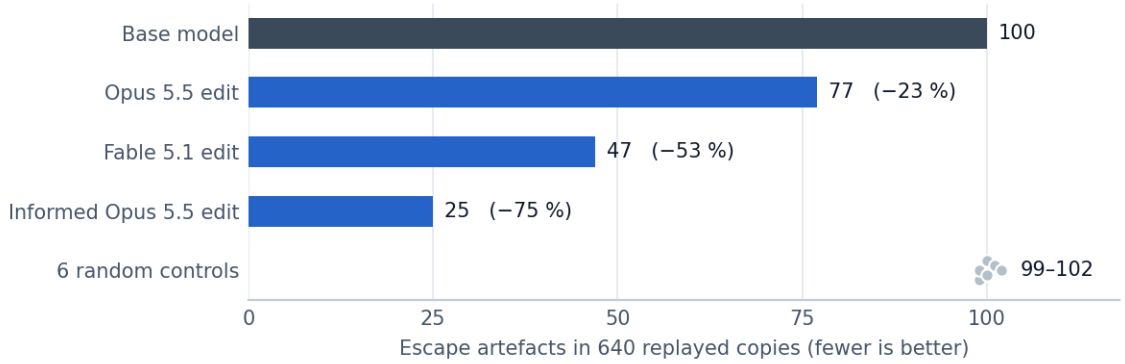


Figure 5. Artefact reductions against each edit's two matched controls. Edits differ in size; this is not a model ranking.

A post-hoc check on 120 test calls shows how narrowly Fable's edit acts (Figure 6). Feeding in the unedited model's own old text (teacher forcing), it changes the most likely next token at 177 of the 303 positions where an escape code was written, but at only 2 of 14,255 ordinary positions, as rarely as two random changes of the same size (1 and 5). A gradient-trained change of the same matrix, rank and size (Section 3.2) changes 133 and 16. Along the unedited text, then, the edit acts almost only at escapes; the next paragraph shows how acting there can still lose working calls.

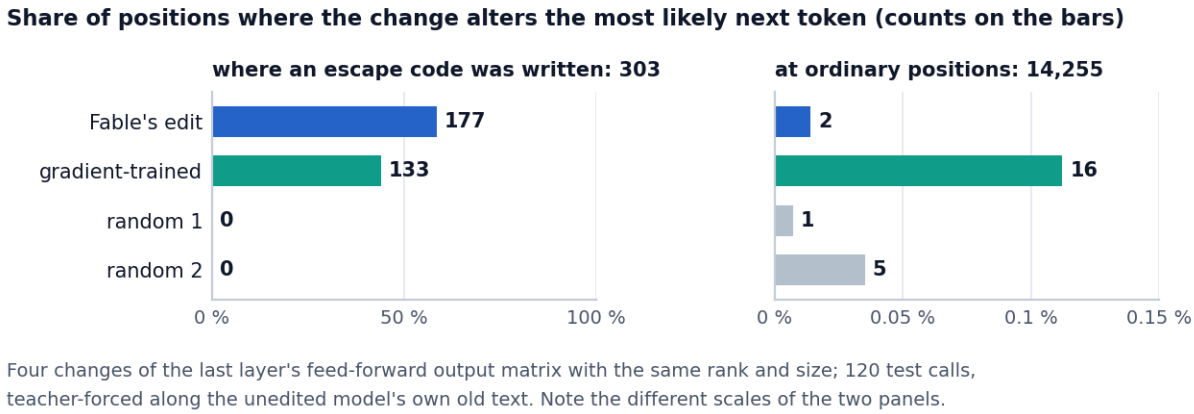


Figure 6. How often four changes of the last layer's feed-forward output matrix, all of the same rank and size, alter the most likely next token on 120 test calls (post hoc, descriptive; note the two scales). The full paper also shows the edit's two detectors and where it moves probability.

Removing escapes can still break a copy (Figure 7b). Of 84 base copies rescued by fallback, Opus, Fable and informed Opus moved 24, 45 and 67 away from fallback, mostly to exact copies; 5, 12 and 24 of these were no longer found. These totals span both draws and include calls outside the working subset, unlike the per-draw preservation losses.

The same generations show what the repair hides. We rescored the same generations post hoc with the fallback switched off. Base accepted calls fall from 443 to 359 of 640, and working calls from 213 per draw to 177 and 174. Opus and Fable preserve this smaller subset, pass all three guards and raise task-averaged acceptance by 4.1 and 7.3 points. They improve 10 and 17 tasks respectively, worsen none, and beat both controls in sign tests. Informed Opus gains 9.3 points but misses the compile guard by 0.1 point. No agent has run without fallback.

Informed Opus named preservation as its main risk: on its own training-repository calls it had lost up to 5 per draw,

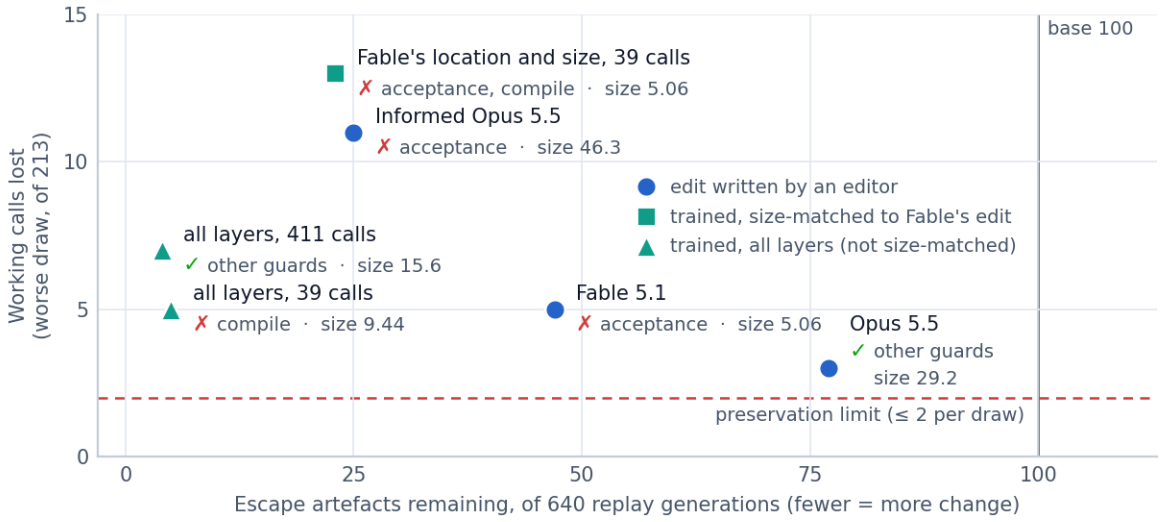
versus 11 here, while its artefact reduction transferred (82 % there, 75 % here). Astra submitted nothing after finding no effect on its own held-aside tasks; both Fable 5.2 runs withdrew after measuring side effects. These single runs do not rank models.

3.2 Comparison with gradient training

In a comparison fixed in advance, we trained LoRA adapters to write correct old text. A call is *labelled* when its recorded old text or unescaped form occurs exactly once in the file: 411 of the 730 training-repository calls the editors could search. One training run used the 39 labelled calls among those Fable read with its requests, matching its location, rank and size. Two others trained all linear layers, one on those 39 calls and one on all 411, without size matching.

The three adapters left 23, 5 and 4 artefacts and lost 13, 5 and 7 working calls in the worse draw (Figure 7a). Matched training beat Fable on artefacts in 13 tasks versus 1 ($p = 0.002$). All three failed preservation; the 411-call run passed the other guards.

(a) Behaviour change and preservation, six candidates



✓ ✗: the other guards, acceptance and compile (see the full paper). Size: Frobenius norm of the weight change.

(b) How a raw copy breaks (synthetic example)

File	`return {"total": total(items), "count": len(items)}`
Unedited model	`return {\"total\": total(items), \"count\": len(items)}` ✓ escaped, rescued by the fallback, applied
Edited model	`return {"total": total(items), "count": len(items)` ✗ raw, but the closing brace is missing: not found

Figure 7. Artefacts versus lost working calls. Only the square matches Fable's location, rank and size. "Other guards" means acceptance and compilation. Below: a synthetic raw-copy failure.

On this target, the edit written by reasoning showed no advantage over training on the same 39 calls. Losing working calls is not specific to gradient-free editing, and all-layer training loses fewer than matched training. Post-hoc rescoring without fallback gains 9–16 acceptance points, but each trained adapter still misses a guard: matched training fails working-call preservation and compilation, all-layer training on 39 calls fails compilation, and training on 411 fails working-call preservation.

3.3 Transfer to the running agent

We selected Fable after seeing replay results, so this choice was exploratory. Served in the competition's vLLM format, with 4-bit weights and 16-bit activations (W4A16), it kept its effect on the replay: artefacts fell from 102 for the quantized base to 46 for the edit; a random adapter left 100. Working-call losses against that base were 7 and 11 per draw, versus 5 in each draw in 16-bit and none for random. This random adapter, Fable's first gate control, is used in all live runs below. Agents with the edit also ran without errors in the competition's scorer, which checks deployment, not improvement.

The first live run was exploratory: on the 115 OOD-43 and OOD-B tasks, artefact rates were 18 % for base, 4 % for edit and 20 % for random (Figure 8). We then froze the edit, analysis and replication rule: lower task rates than both comparators in one-sided sign tests, each $p < 0.05$. The expected edit/base ratio was 0.2–0.6. On OOD-C, rates were 15 %, 3 % and 16 %. The edit beat base in 34 tasks versus 4 ($p = 3 \times 10^{-7}$), and random in 27 versus 7 ($p = 4 \times 10^{-4}$). Live measurements do not retest working-call preservation.

A second planned OOD-C replication enabled *thinking*, reasoning before each action. Artefact rates were 7 % for base, 1 % for edit and 8 % for random. The edit beat base in 16 tasks versus 4 ($p = 0.006$) and random too ($p = 2 \times 10^{-4}$). These were separate experiments, not a planned thinking comparison, so their differing base rates do not measure thinking's effect.

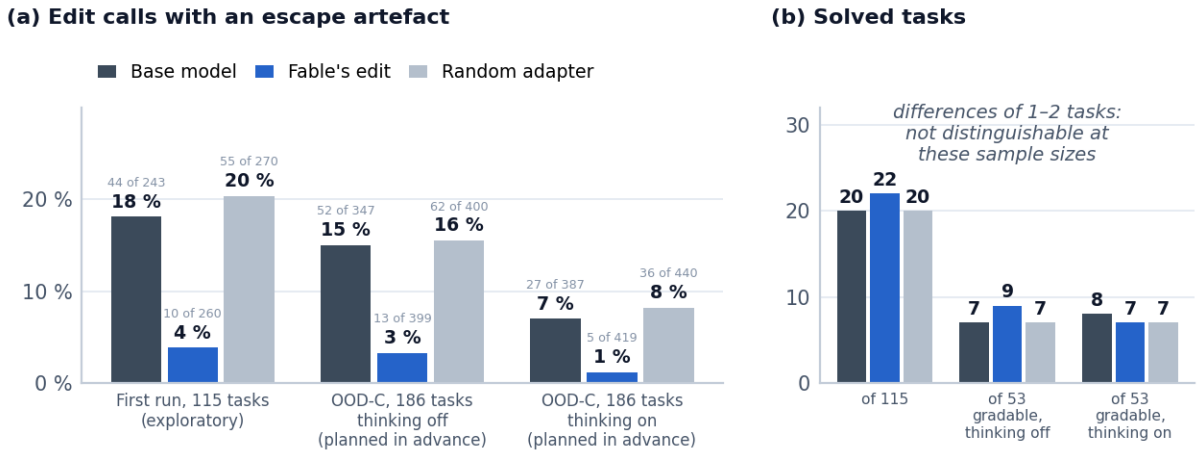


Figure 8. Live artefact rates pool counted calls; the tests compare paired tasks. Solved counts use 115, 53 and 53 tasks; differences of one or two tasks cannot be told apart at these sizes, which does not show equivalence.

3.4 Task outcomes

No solved-task benefit was detected (Figure 8b); these samples cannot establish equivalence. Old-text not-found rates rose for both adapters on OOD-C: without thinking, 27 % for base, 34 % for edit and 32 % for random; with thinking, 25 %, 35 % and 36 %. The cause is untested.

4. Related work and scope

Prior weight-space programming used small networks and a 0.8-billion-parameter model [1]. ROME edits factual associations [6]; steering changes activations [7]. No editor used null-space [9] or covariance-weighted [10] restrictions on preserved inputs. Barry's linear program, which bounds an edit's effect on preserved tokens, and a steering vector both matched the edit of [1] that stops repeated output; Barry names behaviours that change some tokens and not others as the better test [12]; escape artefacts are one. We ran no steering comparison. Tool interfaces also matter [8]. Rendering tool results in the chat template's native, unescaped form raised round-1 acceptance 2.0 points ($p = 0.031$). Registered before generation on round-2 calls, it cut artefacts from 100 to 78. Acceptance moved +0.1 points with fallback, losing 1 and 2 working calls per draw, versus 3.7 points without, losing none (both rules registered). It failed the compile guard under both rules: 10.3 % against base 7.9 % (limit 9.9 %), and 9.0 % against 5.5 % (limit 7.5 %) without fallback. This descriptive comparison has no size-matched random control.

We tested one model, one main target, one run per editor and one per live configuration, with one training seed. We chose the target after counting base test-call artefacts and the deployed edit after replay; planned replications do not undo either selection. Missing traces and file states reduce samples. No all-zero adapter separates serving effects from changed weights. The round-2 rule was committed locally before the first request, pushed during search; replication and training plans were frozen and hashed before running, committed afterwards. In the companion writeup, three edits cut pydantic 2 names in pydantic 1 projects from 0.29 to at most 0.04 of held-out answers; all failed preservation.

A tool's tolerance determines what fixing a behaviour gains. Measure preservation separately from target change, and task outcomes separately from both.

5. Resources

`weightprog` (alpha, Apache-2.0) provides editing, instruments, matched controls and MLX/PyTorch backends; its helper exported these adapters. The companion writeup documents validation and data.

OOD-C supplies offline tasks; the replay set supplies calls, prompts and file states. The study repository holds protocols, audits, replay/editor tools and the full paper. A Kaggle notebook shows token by token where Fable's edit acts, and fits and exports a rank-1 edit.

6. Conclusion

Frontier models' edits changed the target behaviour beyond controls, and one kept the change in live runs, but none met preservation under the competition's harness and no task benefit was detected. Post-hoc rescoring without fallback lets two edits pass: an edit's value depends on the tool's matching rule.

References

1. Takehi, Teoh, Zhang, Khattab. Sparks of Weight-Space Programming. Blog post, 2026.
2. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
3. van Miltenburg, van der Lee, Krahmer. Preregistering NLP Research. NAACL 2021.
4. Badertdinov et al. SWE-rebench. NeurIPS 2025.
5. Badertdinov et al. SWE-rebench V2. 2026.
6. Meng et al. Locating and Editing Factual Associations in GPT. NeurIPS 2022.
7. Turner et al. Steering Language Models With Activation Engineering. arXiv 2023.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025.
10. Meng et al. Mass-Editing Memory in a Transformer. ICLR 2023.
11. Yan et al. Positive-Congruent Training: Towards Regression-Free Model Updates. CVPR 2021.
12. Barry. Can a Linear Program Do Weight-Space Programming?. Blog post, 2026.